Project - Setup

Conda and LLM Tools

This document provides essential setup instructions for the Software Testing course project and exercises. It covers two key components:

- **Conda** — a package manager for creating an isolated Python environment, ensuring project dependencies do not conflict with your system installation.
- **LLM Tools** — an overview of large language model tools needed for the AI review components included in each task sheet.

Conda Installation

We recommend setting up a `conda` environment for your Python projects as it allows you to isolate dependencies and Python versions from your system installation. This helps avoid conflicts between different projects requiring different Python versions.

a) Download and install Conda:

1) Windows:
   - Make sure to read the installation instructions at https://docs.conda.io/en/latest/miniconda.html as the installation process may ask for path modifications impacting the accessibility of Python and Conda commands from your system's command line.
   - Download Miniconda installer from https://docs.conda.io/en/latest/miniconda.html
   - Run the installer (.exe file)
   - Open "Anaconda Prompt" from Start Menu
2) macOS:
   - Install with Homebrew:

     ```
     $ brew install miniconda
     ```
   - Or download installer from https://docs.conda.io/en/latest/miniconda.html
   - Open Terminal
3) Linux:
   - Download installer:

     ```
     $ wget https://repo.anaconda.com/miniconda/\\Miniconda3-latest-Linux-x86
     ```
   - Run:

     ```
     $ bash Miniconda3-latest-Linux-x86_64.sh
     ```
   - Open Terminal

b) Create environment (from Terminal/Anaconda Prompt):

```
$ conda create --name <myenv> python=<3.9>
```

- Replace `<myenv>` with your preferred name
- Specify Python version `<3.9>` as needed

c) Activate environment:

```
$ conda activate <myenv>
```

d) Install required packages:

```
$ conda install package_name
```

or

```
$ pip install package_name
```

LLM Installation

Large Language Models (LLMs) are advanced AI systems trained on vast amounts of text data to understand and generate human-like text. These models can engage in conversations, answer questions, and assist with various tasks through natural language interaction. Modern LLMs can also understand and generate code across multiple programming languages.

Throughout this course, some exercise tasks will include an AI review component where you'll be asked to solve specific parts using LLMs of your choice. While you may use commercial LLM services such as OpenAI's ChatGPT, Anthropic's Claude, or Google Gemini, we would like to highlight two open-source alternatives that provide local access to public models:

a) Ollama (https://ollama.ai/):

- A lightweight, command-line tool optimized for different hardware configurations
- Allows easy loading and running of public models locally
- Due to its hardware optimizations, it might offer better performance on certain systems
- Example:

```
# Install and run deepseek-r1 model
$ ollama run deepseek-r1:7b
```

b) LM Studio (https://lmstudio.ai/):

- User-friendly graphical interface for managing and running language models
- Simplified process for downloading and running models through point-and-click interaction
- Includes built-in chat interface and model management features

For optimal performance on most systems, we recommend using models with 7 billion parameters or less, unless you have sufficient RAM/VRAM available for larger models. Regarding compatibility: Ollama supports Linux, macOS (version 11 and above), and Windows, while LM Studio is compatible with Linux, Windows, and macOS systems with M-series chips.

Submission Templates

Each project phase comes with a dedicated **Markdown submission template** (`phase01_submission.md` through `phase05_submission.md`, and `phaseR_submission.md` for the research task). These templates are structured analogously to the exercise submission sheets and work as follows:

- **Do not modify** section headings or task descriptions — they are part of the template structure.
- **Fill in your answers only where indicated**, replacing placeholder text such as *Your answer here* or `[TODO]`.
- Add screenshots as embedded images using the provided HTML comment hints.
- Reference your `*.py` test files by name or paste relevant excerpts into the code blocks.
- Each answer should be **annotated** with the contributing group member where applicable.

Submit the completed Markdown file together with your `*.py` test files as a `*.zip` to the corresponding folder on StudIP (see the project task sheet for naming conventions and folder structure). Additionally, incrementally add your Python tests to your group's GitLab repository.